```
#include <atomic>
#include <thread>
#include <assert.h>

std::atomic<bool> x,y;
std::atomic<int> z;

void write_x_then_y()
{
    x.store(true,std::memory_order_relaxed);    <-1
    y.store(true,std::memory_order_relaxed);    <-2
}

void read_y_then_x()
{
    while(!y.load(std::memory_order_relaxed));    <-3
    if(x.load(std::memory_order_relaxed))    <-4
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x_then_y);
    std::thread b(read_y_then_x);
    a.join();
    b.join();
    assert(z.load()!=0);    <-5
}
```

This time the assert ❺*can* fire, because the load of `x` ❹ can read `false`, even though the load of `y` ❸ reads `true` and the store of `x` ❶ happens-before the store of `y` ❷. `x` and `y` are different variables, so there are no ordering guarantees relating to the visibility of values arising from operations on each.

Relaxed operations on different variables can be freely reordered provided they obey any happens-before relationships they're bound by (for example, within the same thread). They don't introduce synchronizes-with relationships. The happens-before relationships from listing 5.5 are shown in figure 5.4, along with a possible outcome. Even though there's a happens-before relationship between the stores and between the loads, there isn't one between either store and either load, and so the loads can see the stores out of order.

Figure 5.4. Relaxed atomics and happens-before